

Patroni Internals: A Deep Technical Report for PostgreSQL Specialists

TL;DR

- **Patroni is a Python (95.7% of the repo) agent-per-node HA template for PostgreSQL** that runs one bot alongside each Postgres instance; the bots elect a single leader via a TTL-based leader lock in a Distributed Configuration Store (etcd/etcd3, Consul, ZooKeeper, Exhibitor, Kubernetes API, or an embedded Raft), and each bot independently decides to promote/demote its local Postgres by comparing WAL/LSN positions over a REST API. The current stable release is **v4.1.3 (May 5, 2026)**, supporting PostgreSQL **9.3–18**.
- **The heartbeat is the HA control loop** in `(patroni/ha.py)` (`(run_cycle())` → `(_run_cycle())`), executed every `(loop_wait)` seconds (default 10s, min 1s): the leader renews its lock (TTL `(ttl)`, default 30s, min 20s), and on lock expiry the remaining replicas run a leader race gated by `(maximum_lag_on_failover)`, timeline checks, tags (`(nofailover)`/`(failover_priority)`), `(/sync)` membership, and watchdog health. Split-brain is prevented by demoting on lost lock, optional watchdog self-fencing, `(pre_promote)` fencing scripts, and (for DCS outages) `(failsafe_mode)`.
- **Patroni implements the full lifecycle**: bootstrap (initdb or custom), replica creation (pg_basebackup or wal_e/pgBackRest/Barman via `(create_replica_methods)`), automatic failover, manual failover/switchover (scheduled or immediate), restart/reload/reinit, dynamic configuration in DCS, pause/maintenance mode, synchronous (`(on)`/`(quorum)`/`(strict)`) replication, standby clusters, cascading replication, permanent/member replication slots with synchronization, pg_rewind for rejoining former primaries, a rich REST API (health checks, `(/config)`, `(/switchover)`, `(/metrics)`, etc.), the `(patronictl)` CLI, callbacks, watchdog, and Citus integration.

Key Findings

1. **Architecture is a "bot pattern"**: Postgres never talks to the DCS directly. A per-node Patroni agent mediates between Postgres and the DCS, holding/renewing a leader lock and reconciling local Postgres state to the desired cluster state each loop iteration. The DCS provides the only source of truth and the atomic compare-and-set primitive that guarantees a single leader.
2. **Leader election is cooperative and decentralized**: every member decides only for itself, using atomic CAS operations on the leader key and cross-checking candidate health over REST. Etcd/ZooKeeper/Consul provide native TTL/session expiry; Kubernetes lacks key expiry so Patroni emulates it by timestamping annotation updates.
3. **The codebase is cleanly layered**: `(patroni/ha.py)` (control loop), `(patroni/dcs/)` (pluggable DCS abstraction), `(patroni/postgresql/)` (Postgres lifecycle: bootstrap, config, rewind, slots, sync, callbacks), `(patroni/api.py)` (REST), `(patroni/ctl.py)` (CLI), `(patroni/watchdog/)`.
4. **Data-loss controls are first-class**: `(maximum_lag_on_failover)`, `(synchronous_mode)`/`(synchronous_mode_strict)`/`(quorum)`, `(check_timeline)`, watchdog, and `(failsafe_mode)` each address distinct failure modes.
5. **Recent (3.x→4.x) evolution** added quorum commit, DCS failsafe mode, Citus integration, `(/metrics)`, member-slot retention (`(member_slots_ttl)`, default 30 min, available since Patroni 4.0.0), `(failover_priority)`, `(nostream)`, psycopg3 support, and the `(master)`→`(primary)` terminology migration.

Details

1. How Patroni Works: Architecture, Components, and the HA Model

Agent-per-node model. Patroni is "a template for PostgreSQL HA solutions using Python." Each node runs a Patroni process that manages exactly one PostgreSQL instance. Because "PostgreSQL cannot talk to Etcd directly," Patroni employs a bot alongside each Postgres that talks to the DCS and decides on promotion/demotion. The cluster state (leader identity, members, dynamic config, sync state, failover requests, history, failsafe topology) lives in the DCS under `(/namespace/<scope>/...)` keys.

The HA control loop (heartbeat). The daemon `(patroni.py)`'s `(run())` repeatedly calls `(Ha.run_cycle())` in `(patroni/ha.py)`, which wraps `(_run_cycle())` — the core state machine — in DCS-error handling and logs the returned action string each iteration. `(_run_cycle())` dispatches through these branches:

- **Load cluster from DCS** (`(load_cluster_from_dcs())`); on DCS failure `(handle_dcs_error())` may enter failsafe.
- **Not initialized / no data** → `(bootstrap())` (or `(bootstrap_standby_leader())` for standby clusters), then `(post_bootstrap())`.
- **Postgres not running** → `(recover())`, whose docstring reads *"Handle the case when postgres isn't running."* It may run crash recovery in single-user mode, start as primary if it owns the lock, or trigger `(handle_rewind_or_reinitialize())`.
- **Async action in progress** → `(handle_long_action_in_progress())`.
- **We hold the lock** → `(update_lock())` (which calls `(watchdog.keepalive())` after a successful leader-key update), then `(enforce_primary_role())`, `(process_manual_failover_from_leader())`, `(process_healthy_cluster())`, and sync handling `(process_sync_replication())`.

- **No leader key** → `process_unhealthy_cluster()` → leader race via `is_healthiest_node()` and `acquire_lock()`.
- **We are/should be a replica** → `follow()` / `get_node_to_follow()`; `demote(mode)` where `mode` ∈ {`offline`, `graceful`, `immediate`, `immediate-nolock`}.
- **Housekeeping** every cycle: `touch_member()` (publishes conn URLs, LSN, role, tags to DCS), slot sync, history update, scheduled-restart evaluation, and `watch(timeout)`.

Timing model. Per the Dynamic Configuration docs: `loop_wait` (default 10, min 1) = loop sleep; `ttl` (default 30, min 20) = leader-lock lease; `retry_timeout` (default 10, min 3) = DCS/PG operation retry timeout, where "DCS or network issues shorter than this will not cause Patroni to demote the leader." `primary_start_timeout` defaults to 300s, and "Worst case failover time for primary failure is: `loop_wait + primary_start_timeout + loop_wait`, unless `primary_start_timeout` is zero, in which case it's just `loop_wait`." (Patroni)

Postgres lifecycle management. The `patroni/postgresql/` package (class `Postgresql`) manages initdb, start/stop/restart (`pg_ctl`), promote/demote, `pg_isready`, control data parsing, GUC validation, `primary_conninfo`/`primary_slot_name` generation, checkpointing, and role/state tracking.

2. Language, Dependencies, Code Structure, and Testing

Language & Python version. Written in Python (95.7% Python, 2.5% Gherkin for behavioral tests). Notably, the current `setup.py` on `master` contains no `python_requires` argument; supported versions are declared only via trove classifiers ranging from **Python 3.6 through 3.14**. Release notes record that "Version 3.0.0 is the last release supporting Python 2.7" and "Version 3.0.2 dropped support of Python older than 3.6," with 3.7 cited as the floor when 2.7 was dropped. The published wheel is a generic `patroni-4.1.3-py3-none-any.whl`.

Key dependencies (`requirements.txt`): `urllib3>=1.19.1,!=1.21`, `PYAML`, `click>=4.1`, `prettytable>=0.7`, `python-dateutil`, `cryptography>=1.4`, `psutil>=2.0.0`, `ydiff>=1.2.0,<1.5,!=1.4.0,!=1.4.1` (used by `patronictl edit-config` diffs), `boto3` (AWS). DCS-specific: `python-etcd>=0.4.3,<0.5` (`etcd`), `py-consul` (replacing the unmaintained `python-consul`), `kazoo>=1.3.1` (ZooKeeper/Exhibitor), `kubernetes` client, `pysyncobj>=0.3.8` (Raft). Psycpg is intentionally *not* a hard dependency: users choose `psycpg[binary]>=3.0.0` (`psycpg3`), `psycpg2>=2.5.4`, or `psycpg2-binary` via extras. (GitHub + 4)

Repository layout (top-level `patroni/` package):

- `patroni/ha.py` — HA control loop and leader-race logic (`Ha` class, `MemberStatus`, `Failsafe`).
- `patroni/dcs/` — DCS abstraction: `__init__.py` (`AbstractDCS`, `Cluster`, `Leader`, `Member`, `SyncState`, `Status`, `Failover`, `TimelineHistory`, `get_dcs()`), plus `etcd.py`, `etcd3.py`, `consul.py`, `zookeeper.py`, `exhibitor.py`, `kubernetes.py`, `raft.py`.
- `patroni/postgresql/` — `__init__.py` (`Postgresql`), `bootstrap.py`, `config.py`, `rewind.py`, `slots.py`, `sync.py`, `callback_executor.py`, `cancellable.py`, `connection.py`, `postmaster.py`, `validator.py`, `misc.py`, and `mpp/` (Citux).
- `patroni/api.py` — REST API (`RestApiHandler`, `RestApiServer`).
- `patroni/ctl.py` — `patronictl` CLI (built on `click`).
- `patroni/watchdog/` — `base.py` + `linux.py`.
- `patroni/config.py`, `patroni/global_config.py`, `patroni/tags.py`, `patroni/validator.py`, `patroni/scripts/` (`aws.py`, `wale_restore.py`, `barman/`).
- Entry points: `patroni.py`, `patronictl.py`, `patroni_raft_controller.py`.

Testing. Unit tests under `tests/` (pytest, coverage tracked via Codecov), behavioral acceptance tests under `features/` (Gherkin/behave), a Jepsen harness under `jepsen/`, type stubs under `typings/` with pyright (`pyrightconfig.json`), and CI via GitHub Actions (`tests.yaml`). Tests run against Python up to 3.14.

3. Implemented Actions/Operations

- **Bootstrap:** `initdb` by default, or a custom `bootstrap.method` script (e.g., `barman`, `pgBackRest`) with `--scope`/`--datadir` supplied (suppressible via `no_params`), optional `recovery_conf`, and `keep_existing_recovery_conf`. Only the node that wins the `initialize` lock bootstraps.
- **Replica creation:** `pg_basebackup` default; custom ordered `create_replica_methods` (`wal_e`, `pgBackRest`, `Barman`); `clonefrom`-tagged replicas preferred as basebackup source; `no_leader`, `keep_data`, `no_params` flags.
- **Leader election / automatic failover:** TTL lock race (detailed in §4).
- **Manual failover** (works on unhealthy clusters; requires `candidate`) and **switchover** (healthy clusters; requires `leader`; supports scheduling). Failover can go to an async node when sync standbys are unhealthy.
- **Promotion/demotion:** automatic on race win / lock loss; `demote()` modes `offline`/`graceful`/`immediate`/`immediate-nolock`.

- **Restart** (immediate/scheduled/pending-only/by-role/(`--any`)/version-gated), **reload** (SIGHUP equivalent), **reinitialize** (`reinit`), optionally (`--from-leader`) and (`--force`).
- **Dynamic configuration** in DCS (`/config`) (loop_wait, ttl, retry_timeout, PG parameters, slots) vs. local config: `pending_restart` flag with `pending_restart_reason` diff for restart-requiring GUCs; reload for others.
- **Pause / maintenance mode:** detaches Patroni from managing Postgres state (details in §6).
- **Synchronous replication management:** `synchronous_mode` (`off`)/(`on`)/(`quorum`), `synchronous_mode_strict`, `synchronous_node_count` (default 1), `maximum_lag_on_syncnode`; `/sync` DCS key.
- **Standby clusters & cascading replication** via `standby_cluster` and `replicatefrom`.
- **Tags:** `nofailover`, `noloadbalance`, `clonefrom`, `nosync`, `nostream`, `replicatefrom`, `failover_priority`, `sync_priority` (plus arbitrary custom tags queryable via REST).
- **Watchdog self-fencing, callbacks** (`on_start`)/(`on_stop`)/(`on_restart`)/(`on_reload`)/(`on_role_change`), `pre_promote` and `before_stop` scripts.
- **patronictl** commands: `list`, `topology`, `history`, `dsn`, `query`, `switchover`, `failover`, `restart`, `reload`, `reinit`, `pause`, `resume`, `edit-config`, `show-config`, `flush`, `remove`, `version`.

4. How Promotion Is Negotiated (Leader Election in Depth)

The DCS and the leader lock. Patroni supports etcd (v2 and v3/`etcd3`), Consul, ZooKeeper, Exhibitor, Kubernetes, and an embedded Raft (`pysyncobj`, via `patroni_raft_controller`). `patroni/dcs/_init_.py` defines `AbstractDCS` with abstract primitives: `attempt_to_acquire_leader()`, `take_leader()`, `update_leader()`, `delete_leader()`, `write_sync_state()`, `set_config_value()`, `initialize()`, `watch()`, etc. The leader key is created atomically with value = node name and TTL = `ttl`; "the key must be created atomically. In case the key already exists it should not be overwritten." Etcd/ZooKeeper/Consul rely on native TTL/session expiry; Kubernetes has no expiry so Patroni times leader-annotation updates and treats the key as expired once `leader_observed_time + ttl` passes. (Patroni)

Lock renewal and expiry. The leader renews its lock each loop via `update_lock()` → `AbstractDCS.update_leader()` (also updating `/status`, `/optime/leader`, and `/failsafe`). If renewal fails for reasons other than value/version mismatch, the primary is demoted before the key expires to avoid split-brain. When the key expires, replicas detect the unlocked cluster and start the leader race. (Patroni)

The leader race. In `process_unhealthy_cluster()`, each replica evaluates `is_healthiest_node()` → `_is_healthiest_node()` ("tries to determine whether I am healthy enough to become a new leader candidate"). Checks include:

- **Replication lag:** `is_lagging(wal_position)` rejects candidates lagging more than `maximum_lag_on_failover` bytes.
- **Cross-member LSN comparison via REST:** `fetch_node_status()` (`fetch_nodes_statuses()`) GET each member's `/patroni`, producing `MemberStatus` (fields `reachable`, `in_recovery`, `wal_position` = max of replayed/received location, `timeline`, `tags`, `watchdog_failed`). A node that finds any reachable eligible member with a higher LSN steps aside.
- **Timeline:** if `check_timeline` is true, a candidate on an older timeline is rejected.
- **Tags:** `nofailover` (equivalently `failover_priority: 0`) excludes a node; higher `failover_priority` is preferred. `MemberStatus.failover_limitation()` returns the reason a node cannot promote.
- **Watchdog:** a member requiring but lacking a functional watchdog is disqualified.
- **Sync membership:** in `synchronous_mode`, only nodes listed in `/sync` are eligible (the leader race is run only against known synchronous nodes).

The first healthy node to win the atomic CAS on the leader key promotes; others reconfigure to follow it.

Fencing / split-brain prevention. (a) **Demote on lost lock** — the primary demotes itself if it cannot renew. (b) **Watchdog** — `watchdog.mode` (`off`)/(`automatic`)/(`required`); Patroni arms the watchdog before promotion and sends keepalives after each successful lock update. Per the Watchdog docs: "By default Patroni will set up the watchdog to expire 5 seconds before TTL expires. With the default setup of loop_wait=10 and ttl=30 this gives HA loop at least 15 seconds (ttl - safety_margin - loop_wait) to complete"; setting `safety_margin: -1` makes the watchdog timeout `ttl // 2` for an absolute guarantee. (c) **pre_promote** — a fencing script run after acquiring the lock but before promotion; a non-zero exit aborts promotion and releases the leader key. (d) A guard prevents a returning ex-primary from "happily acquiring" a leader key and discarding a newer promotion.

Switchover vs. failover negotiation. A **switchover** (healthy cluster) requires the `leader` field, optionally a `candidate`, and can be scheduled; the leader steps down and, if no candidate is named, all eligible nodes race. A **failover** requires a `candidate`, can run on a leaderless/unhealthy cluster, and cannot be scheduled. In synchronous mode a switchover is only allowed to a node listed in `/sync`; a manual failover in a leaderless cluster bypasses sync/lag/timeline checks (with explicit data-loss warnings). Requests flow through `POST /switchover` / `POST /failover` and `DCS.manual_failover()`, handled in `_run_cycle()` by `process_manual_failover_from_leader()` (leader side) and `manual_failover_process_no_leader()` (leaderless side).

5. How a New Replica Is Created

Default path — `pg_basebackup`. Used when `create_replica_methods` is empty; the base backup is taken from the leader unless a `clonefrom`-tagged replica exists, in which case that replica is the source. A `basebackup` config section may pass long options (e.g., `max-rate`, `checkpoint: fast`, `waldir`).

Custom replica creation methods. An ordered `create_replica_methods` list (e.g., `wal_e`, `pgbackrest`, `barman`, then `basebackup`); Patroni stops on the first method returning 0. Each method receives `--scope`, `--datadir`, `--role` (always `replica`), and `--connstring`; special flags `no_leader` (allow creation without a running leader — for backup restores), `keep_data`, `no_params`. If all methods fail, they are retried next loop.

Recovery configuration. For PostgreSQL ≤ 11 Patroni writes `recovery.conf`; for PG ≥ 12 it writes recovery GUCs into `postgresql.conf`/`postgresql.auto.conf` and creates `standby.signal` (or `recovery.signal`). `primary_conninfo` and `primary_slot_name` are generated from member/leader info.

Replication slots. With PostgreSQL ≥ 11, Patroni maintains physical replication slots on all nodes that could become leader, so replicas retain needed WAL. When a member's key expires, its slot is dropped after `member_slots_ttl` (default 30 min; set 0 for immediate removal; feature requires PG ≥ 11). **Permanent slots** (`slots:` config, physical or logical) survive switchover/failover and are created where missing; permanent physical slots on all nodes have positions advanced every `loop_wait`. **Permanent logical slots are copied from the primary to standbys (with restart) and then advanced**, and are synchronized only from primary/standby_leader to replicas. `patroni/postgresql/slots.py` (`SlotsHandler`, `SlotsAdvanceThread`) implements this; `nostream` disables slot copying/sync on a node and its cascade.

Reinitialization (`reinit`). Allowed only on replicas: removes PGDATA and re-runs the replica creation method; `--force` overrides a recovery loop, `--from-leader` forces basebackup from the leader.

pg_rewind for rejoining a former primary. `patroni/postgresql/rewind.py` (`Rewind`) runs `pg_rewind` when `use_pg_rewind` is set and a former primary's timeline diverged. It requires data checksums and/or `wal_log_hints`. It verifies the leader is not in recovery (`check_leader_is_not_in_recovery`), reads timeline history, uses `--restore-target-wal` or fetches missing WAL via `restore_command`, and parses `pg_waldump` output to locate the checkpoint end (needed because the `pg_waldump` error format changed in PG16). `remove_data_directory_on_rewind_failure` (default false) re-clones on rewind failure; `remove_data_directory_on_diverged_timelines` re-clones when timelines diverge and the old primary cannot stream. A dedicated `rewind` user/credentials can be configured (PG ≥ 11).

Cascading replication. `replicatefrom: <member>` makes a replica stream from another replica rather than the leader; `_get_node_to_follow()` resolves the upstream, falling back to the leader if the named source is unavailable.

6. What Else Patroni Can Do

REST API (`patroni/api.py`) — used by the leader race, patronictl, and load balancers:

- Health checks:** `GET /`, `/primary`, `/read-write` (200 only on primary-with-lock); `/standby-leader`; `/leader` (has lock regardless of primary/standby); `/replica` (running replica without `no_loadbalance`), with `?lag=<bytes|16MB>` and `?tag_key=value` filters; `/read-only`; `/synchronous` (`/sync`), `/read-only-sync`; `/quorum`, `/read-only-quorum`; `/asynchronous` (`/async`) with lag filter; `/health`; `/liveness` (heartbeat alive) and `/readiness` (caught-up) for Kubernetes probes. [readthedocs + 2](#)
- Monitoring:** `GET /patroni` (full JSON status, used during the leader race), `GET /metrics` (Prometheus format: `patroni_primary`, `patroni_xlog_location`, `patroni_postgres_running`, `patroni_pending_restart`, `patroni_is_paused`, `patroni_postgres_state`, `patroni_failsafe_mode_is_active`, etc.). [readthedocs](#)
- Cluster status:** `GET /cluster` (topology, lag, scheduled switchover), `GET /history`. [readthedocs](#)
- Config:** `GET/PATCH/PUT /config` (dynamic config; PATCH with `null` removes a key). [readthedocs](#)
- Control:** `POST/DELETE /switchover`, `POST /failover`, `POST/DELETE /restart`, `POST /reload`, `POST /reinitialize`, `POST /failsafe`, `POST /citus`.

DCS integrations. etcd v2/v3, Consul (with optional service registration and tags), ZooKeeper, Exhibitor, Kubernetes, and embedded Raft. On **Kubernetes**, Patroni operates without a separate DCS by storing leader/config in Endpoints (recommended;

`kubernetes.use_endpoints=true`, requires `pod_ip`) or ConfigMaps annotations; it sets pod role labels (`role=primary`, customizable via `role_label`, `leader_label_value`, etc.) and emulates leader-key expiry via timestamped annotation updates (idea derived from `client-go` leaderelection). This is the operator-style deployment used by Spilo and the Zalando postgres-operator. [Patroni](#)

DCS failsafe mode (`patroni/ha.py` `Failsafe`). Enabled only via dynamic config (`failsafe_mode: true`). Introduces a permanent `/failsafe` DCS key listing all known members, maintained by the leader. During a DCS outage, if the leader cannot update its lock, it POSTs `/failsafe` to every member in the topology and continues as primary only if **all** members acknowledge (the requirement to reach *all* members — not a quorum — is deliberate, because Patroni's node distribution across AZs is not guaranteed). Replicas treat an incoming

`/failsafe` request as proof the primary is alive (cached for `(ttl)`) and refrain from racing. A member may only become leader if it appears in `/failsafe`.

Pause / maintenance mode (`(patroni/ha.py)`, `(docs/pause.rst)`). Patroni stops changing Postgres state: it keeps updating member keys and the leader lock, but will not auto-promote replicas, will not start/stop Postgres, and releases (rather than reclaims) the lock if the lock owner stops being primary. Manual unscheduled restart/failover/reinit are permitted; scheduled actions are not; manual switchover requires a named candidate; on detecting parallel primaries it warns but does not demote. Used for major upgrades and corruption recovery.

Standby clusters (`(docs/standby_cluster.rst)`) provide cross-DC cascading: a "standby leader" holds the DCS lock and replicates from a remote primary while local cascade replicas follow it. Multiple source hosts add `(target_session_attrs=read-write)`. A warning notes member-name collisions between primary and standby clusters can silently satisfy `(synchronous_standby_names)` from the wrong connection — a data-loss hazard. `(readthedocs)`

Citus support (since 3.0; `(patroni/postgresql/mpp/)`). Patroni manages a Citus coordinator (group 0) and worker groups (1,2,...), registers workers/secondaries in `(pg_dist_node)` (including replicas without `(no_loadbalance)`), sets `(citus.local_hostname)`, and coordinates worker switchovers through the coordinator via REST. `(Patroni)`

Callbacks & scripts. `(CallbackExecutor)`/`(CallbackAction)` run `(on_start)`/`(on_stop)`/`(on_restart)`/`(on_reload)`/`(on_role_change)` scripts (receiving action, role, scope) — commonly used for virtual-IP/DNS management; `(on_reload)` was fixed so it cannot kill other callbacks. `(pre_promote)` (fencing) and `(before_stop)` (synchronous pre-shutdown) are distinct from callbacks. `(Patroni)`

Logical replication slots are handled as permanent slots synchronized from the primary; `(nostream)` disables their sync on a node and its cascade; standby clusters cannot create logical slots (they are ignored there).

Notable recent features (as of the 2026 main branch). Quorum commit (`(synchronous_mode: quorum)`), DCS failsafe mode, Citus integration and secondary registration in `(pg_dist_node)`, `(/metrics)` Prometheus endpoint, `(member_slots_ttl)` retention, `(failover_priority)` and `(nostream)` tags, JSON log format, `(pending_restart_reason)` diffs, the `(patroni_barman)` contrib for Barman bootstrap/replica/callback, psycopg3 support, py-consul replacing python-consul, the `(master)`→`(primary)` terminology migration (REST/labels/patronictl), and hardening for etcd v3.6.9/v3.5.28/v3.4.42 security fixes and Kubernetes 403/401 transient errors. A README advisory warns that on Python 3.11+ with `(vm.overcommit_memory=2)`, a CPython thread-start bug can hang the REST API; releases 4.1.1+/4.0.8+ mitigate by starting threads early, and tuning `(MALLOC_ARENA_MAX=1)`, `(thread_stack_size)` (Patroni default 512 kB), and `(thread_pool_size)` (default 5, "sufficient for three-node clusters") is recommended. `(Patroni + 2)`

Recommendations

Staged adoption and configuration guidance for a senior DBA:

1. **Start by mapping durability requirements to mode.** For zero-RPO workloads, enable `(synchronous_mode: quorum)` (PG ≥ 10) with `(synchronous_node_count)` sized to your standby count, and set `(maximum_lag_on_failover: 0)`. For availability-first workloads, keep async with a modest `(maximum_lag_on_failover)` (e.g., 1 MB). Threshold to change: if write-latency variance from sync replication is unacceptable, drop from `(strict)` to `(on)`, or from `(on)` to async with `(check_timeline: true)`.
2. **Always deploy split-brain protection in layers.** Enable the Linux `(softdog)` watchdog in `(mode: required)` on bare-metal/VM nodes; set `(safety_margin: -1)` (watchdog = `(ttl//2)`) if you need an absolute guarantee, and correspondingly raise `(ttl)` / lower `(loop_wait)` + `(retry_timeout)`. Add a `(pre_promote)` fencing script where external fencing (SAN, VIP) is available. `(readthedocs)`
3. **Enable `(failsafe_mode: true)`** if your DCS shares infrastructure with the database (especially Kubernetes-API-backed DCS), after confirming all members can authenticate to each other's REST API — otherwise the first real DCS outage will demote the primary. Verify by inspecting `(failsafe_mode_is_active)` in `(GET /patroni)`.
4. **Configure `(use_pg_rewind: true)`** with `(--data-checksums)` (at initdb) or `(wal_log_hints=on)`, and a dedicated `(rewind)` user (PG ≥ 11). Leave `(remove_data_directory_on_rewind_failure)` at its default `(false)` initially; only enable it once you have continuous WAL archiving/PITR so an errant re-clone cannot lose the only copy of un-replicated data. `(Patroni)`
5. **For static topologies, define permanent physical slots** matching member names (or tune `(member_slots_ttl)`) to prevent WAL recycling while a replica is briefly down. For large clusters, offload cloning with a `(clonefrom)` replica and/or pgBackRest/Barman `(create_replica_methods)` to avoid loading the primary.
6. **Wire the REST API into your load balancer** (`(/primary)` for writes, `(/replica?lag=...)` for reads) and scrape `(/metrics)`; on Kubernetes use `(/readiness)` and `(/liveness)` probes tuned so probes begin failing around leader-key expiry.
7. **Operationally,** use `(patronictl pause)` before any manual Postgres maintenance or Patroni upgrade (a daemon restart bounces Postgres and can trigger failover), prefer `(switchover)` over `(failover)` on healthy clusters, and manage all runtime config through `(patronictl edit-config)` / `(PATCH /config)` — never expect `(bootstrap.dcs)` edits to take effect post-bootstrap.

- **Version/date currency:** findings reflect Patroni **v4.1.3** (latest release May 5, 2026) and the `master`/`latest` docs; specific line numbers in `ha.py` are approximate (drawn from the v3.3.10-tagged module reference, which matches master's method layout) and may shift between commits.
- **No enforced `python_requires`:** the current `setup.py` omits `python_requires`; the *effective* minimum inferred from classifiers and release notes is Python 3.6–3.7. Treat "3.7+" as the practical floor rather than a hard-guaranteed contract, and note the Python 3.11+ memory-pressure REST-API hang advisory.
- **Synchronous mode is not absolute:** even `synchronous_mode_strict` can lose transactions if a backend is canceled while awaiting replication ack; simultaneous primary+sync-standby failure can still promote a node missing transactions.
- **RAFT DCS is deprecated** (since 3.0) — maintained on a best-effort basis; not recommended for new production clusters.
- **Standby-cluster member-name collisions** with the primary cluster can silently break synchronous guarantees (a documented data-loss scenario).
- **Kubernetes leader election emulates TTL** via annotation timestamps rather than native key expiry; ConfigMap-based leader changes take at least two updates, so Endpoints mode is preferred where available.
- Some code-internals detail (exact method line numbers, private helper names) is grounded in the rendered API docs and source snippets rather than a full line-by-line read of every module on `master`.